CHAPTER 34

Error Contracts, Validation and Idempotency

In this chapter

Introduction: The Refund That Happened Twice . . . 3
34.1 Core Concepts . . . 4
34.2 The Three Documented Error Kinds . . . 6
34.3 The Error String Is a Prompt . . . 7
34.4 What Must Never Be in an Error String . . . 8
34.5 The Call You Deliberately Did Not Run . . . 8
34.6 Validation at the Call Site . . . 9
34.7 Authorising the Caller, Never the Request . . . 10
34.8 Two Layers Called Retry . . . 11
34.9 The Key Has to Come From Upstream of the Model . . . 12
34.10 A Window Is a Bet . . . 13
34.11 Partial Failure in a Fan-out . . . 13
34.12 The INVALID_JSON Contract . . . 14
34.13 Where This Sits in a Real System . . . 14
34.14 Analogies, and Where They Break . . . 16
34.15 Worked Example: One Tool Call, Five Gates . . . 16
34.16 Code Walkthrough: The Validator You Did Not Write . . . 17
34.17 Mini Project: The Five Gates, Measured . . . 19
34.18 Experiment One: The Gate That Knows What the Grammar Missed . . . 20
34.19 Experiment Two: Duplicated Effects at Two Layers . . . 21
34.20 Experiment Three: One Fan-out, Five Outcomes . . . 22
34.21 Running It: Tests, Deployment and Evaluation . . . 23
34.22 Experiment Four: The is_error String, Five Ways . . . 24
34.23 Hands-on Labs . . . 25
34.24 Lab Solutions . . . 26
34.25 Best Practices . . . 27
34.26 Common Mistakes . . . 28
34.27 Performance Considerations . . . 30
34.28 Security Considerations . . . 31

34.29 Real Production Examples 32
34.30 Certification Notes 32
34.31 Twenty Practice Questions 33
34.32 Answers 34
34.33 Ten Practical Exercises 36
34.34 Five Challenge Labs 36
34.35 Ten Interview Questions 37
34.36 Summary 37
34.37 Glossary 39
34.38 Further Reading 40

Part 6 of 13 · Chapter 34 of 64. Next: Chapter 35 — Tool Chaining and Orchestration.

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Treat `is_error: true` as a channel addressed to the model rather than to your logs, and write a string that helps the next turn instead of repeating the last one.
2. Name the three documented error kinds — tool execution error, invalid tool name, server tool error — and say which of the three you are not responsible for.
3. Build a **call-site validator** from the same schema the model was given, re-checking exactly the constraints Chapter 32 measured as unenforced.
4. State the difference between authorising a **caller** and authorising a **request**, and place the check where a prompt cannot reach it.
5. Explain why `allowed_callers` is a routing field and not an authorisation boundary, and what the documentation does and does not promise about it.
6. Distinguish **retry at the HTTP layer** from **retry at the turn layer**, and predict which one can duplicate a side effect.
7. Make an effect idempotent with a key your loop mints, a scope you choose and a de-duplication window you can defend.
8. Return a legal user message when only some of a parallel batch succeeded, including for calls you deliberately refused to run.

NOTE

Prerequisites

Chapter 17 for fine-grained tool streaming, the `INVALID_JSON` wrapper and the `is_error` contract that goes with it. **Chapter 25** for the four guardrail layers and the rule that only the action layer prevents harm. **Chapter 28** for the `Message` object, the seven stop reasons and the unresolved `server_tool_use` block. **Chapter 29** for the thirteen documented outcomes, retry classification, jitter, budgets, and the two facts this chapter has to pay for: a client-side timeout does not cancel the generation, and no idempotency key is documented on `POST /v1/messages`. **Chapter 31** for the client loop, the continuation rules and tool results as untrusted input. **Chapter 32** for the declared-versus-enforced audit — 16 constraints declared, 13 enforced, 3 stripped into the description. **Chapter 33** for the description as an affordance, and for pairing a comparison rather than reporting two unpaired proportions.

Deferred by name: **orchestration across many tools and turns** is **Chapter 35**; **tools whose error contracts you did not write** arrive with MCP in **Part 7 (Chapters 36–40)**; **the agent loop's own failure and recovery behaviour** is **Chapters 47–51**; **tracing a duplicated effect back to the turn that caused it** is **Chapter 57**; **prompt injection through a tool result** is **Chapter 58**.

Introduction: The Refund That Happened Twice

The incident report is four lines long and every line is technically correct.

A customer asked for a refund. The support agent's assistant called `payments_refund` with the right account, the right amount and the right reference. The refund went through. The application then posted the `tool_result` back to the Messages API to let Claude write the confirmation, and that HTTP request

timed out after twenty seconds. The job runner did what job runners do: it replayed the task from its last durable checkpoint, which was the customer's message. Claude was sampled again, produced a fresh `tool_use` block with a fresh `toolu_` id, and the loop refunded the money a second time.

Nobody wrote a bug. Every component behaved as documented.

The retry policy was the one Chapter 29 built — a classifier, exponential backoff, full jitter, `retry-after` honoured, a fleet-wide budget. It is a good policy and it had nothing to say about this, because it governs one HTTP request and the duplicated effect happened one layer up. The tool schema was the one Chapter 32 built, and it declared an `idempotency_key` with a `pattern` of `^[a-f0-9]{32}$`. Claude filled it in. On the replay, Claude filled it in again — with a different value, because nothing in the system asked it to remember.

And the error string that the timeout eventually produced, the one that went back to the model as a `tool_result` with `is_error: true`, read in full:

```
Error: request failed
```

So the model had no idea whether the money had moved, and neither did anyone else until the customer rang back.

This chapter is about the seam that incident fell through. Chapters 31, 32 and 33 built the happy path: a loop that runs tools, a schema that shapes their inputs, and words that get the right tool chosen. Each of those chapters deferred the same set of questions to this one, five times between them. What do you send back when the tool fails? What re-checks the values the grammar was never going to check? Who is allowed to call this? And what happens when you do the whole thing twice?

The answers are not exotic. They are ordinary distributed-systems engineering, wearing an unfamiliar hat: the client that retries is a language model, and the error message is a prompt.

There are two ideas worth carrying out of this chapter before any of the detail.

The first is that `is_error: true` **addresses the model, not your logs**. The documentation says so directly — *"Write instructive error messages. Instead of generic errors like* `failed`*, include what went wrong and what Claude should try next, e.g.,* `Rate limit exceeded. Retry after 60 seconds.` *This gives Claude the context it needs to recover or adapt without guessing."* Everything Chapter 33 argued about descriptions applies to failures, because a failure message is a prompt written at the worst possible moment and read immediately.

The second is that **the two retry layers look identical from the outside and are not**. Retrying the sampling call cannot duplicate an effect: the model never executes anything, so a re-sent request produces a fresh `tool_use` block your loop has not run. Retrying the *turn* can, because your loop runs the tool and then posts the result, and the post is the part that failed. §34.19 measures the difference: **0.0 duplicated effects per thousand logical requests at the HTTP layer, and 31.65 per thousand at the turn layer** under the same failure rate, on the same code, with the same word — *retry* — describing both.

34.1 Core Concepts

Every term below is used later in the chapter, so each one is defined before it appears.

DEFINITION

Error contract

The agreement between your tool and the model about how failure is communicated: which failures produce a `tool_result` with `is_error: true`, what the `content` string contains, and what the model is expected to do next. It is a contract in the ordinary sense — both sides rely on it — and, unlike the input schema, nothing validates it.

DEFINITION

Call site

The point in your code where a tool's effect actually happens: the function that moves the money, sends the email or writes the row. Chapter 25 called this the **action layer**. It is the last place where anything can be prevented, and the only place where prevention counts.

DEFINITION

Call-site validation

Re-checking the model's tool input against your original schema, in your code, immediately before the effect. Distinct from schema validation at the API, which Chapter 32 showed enforces the supported subset and silently drops the rest.

DEFINITION

Idempotency

The property that performing an operation more than once has the same observable effect as performing it once. Reading a row is naturally idempotent; refunding a card is not, and has to be made so.

DEFINITION

Idempotency key

A value that identifies the *logical* operation rather than the *physical* request, so that a second delivery of the same operation can be recognised and suppressed. Its usefulness is exactly the stability of its source.

DEFINITION

De-duplication window

The period during which your system remembers an idempotency key and will suppress a repeat. Outside the window the key is forgotten and a replay executes again.

DEFINITION

Logical request

One intent that entered your system from outside: a customer clicking *Refund*, a webhook arriving, a row appearing on a queue. One logical request may cause many HTTP requests, several assistant turns and any number of retries. It is the unit an idempotency key should be scoped to.

DEFINITION

HTTP-layer retry

Re-sending one `POST /v1/messages` after a retryable outcome, as Chapter 29's policy and the SDKs' `max_retries` do. It re-samples the model. It never re-executes your tool.

DEFINITION

Turn-layer retry

Re-running an entire assistant turn — sampling, tool execution, continuation — usually because a job runner, queue or user replayed the work. It re-executes your tool by construction.

DEFINITION

Caller

The identity on whose behalf a tool runs. In this chapter it always comes from your session, never from the tool input, because the tool input is written by a component that has read untrusted text.

34.2 The Three Documented Error Kinds

The `handle-tool-calls` page names three, and the distinction matters because you are responsible for two of them and explicitly not responsible for the third. All three were verified on 3 August 2026.

Tool execution error. *"If the tool itself throws an error during execution (for example, a network error when fetching weather data), you can return the error message in the* `content` *along with* `is_error: true` *."* The documented example content is `ConnectionError: the weather service API is not available (HTTP 500)`, and the documented consequence is that Claude incorporates it into its answer to the user.

Invalid tool name. Despite the heading, this covers invalid *use*: *"If Claude's attempted use of a tool is invalid (for example, missing required parameters), it usually means that there wasn't enough information for Claude to use the tool correctly."* Two remedies are given, and they are in order of preference. The first is Chapter 33's: better `description` values. The second is to continue the conversation with an `is_error` result, whereupon *"Claude will try to use the tool again with the missing information filled in"*. The documentation adds a number worth remembering: **if a tool request is invalid or missing parameters, Claude will retry 2–3 times with corrections before apologising to the user.** That is a budget, and a bad error string spends it on guesses.

Server tool errors. *"When server tools encounter errors (for example, network issues with Web Search), Claude will transparently handle these errors and attempt to provide an alternative response or*

explanation to the user. Unlike client tools, you do not need to handle `is_error` *results for server tools."* For web search specifically the documented error codes are `too_many_requests`, `invalid_input`, `max_uses_exceeded`, `query_too_long` and `unavailable`.

That third kind closes a loop opened in Chapter 31: you never construct a `tool_result` for a server tool, and you do not construct its error result either. What you *do* still owe the server tool is the continuation rule — when an unresolved `server_tool_use` block is present, your user message must contain nothing but `tool_result` blocks. §34.20 reproduces the 400 offline.

Kind	Who reports it	What you send	What Claude does
Tool execution error	You	`tool_result`, `is_error: true`, an instructive string	Adapts or explains to the user
Invalid tool use	You	`tool_result`, `is_error: true`, naming what was missing	Retries 2–3 times with corrections
Server tool error	Anthropic	Nothing	Handles it and continues, or explains

> WARNING
>
> **The fourth kind, which is not an error**
>
> A refusal is not in this table. Chapter 25 established that a refusal is a successful HTTP 200 with `stop_reason: "refusal"`, and that `content` may be empty. It is not a tool error, it does not carry `is_error`, and no `tool_result` will fix it. Branch on `stop_reason` before you look at anything else.

34.3 The Error String Is a Prompt

Chapter 21 established that the system prompt is the operator channel, and Chapter 31 established that a `tool_result` is the *untrusted* channel — the documentation is explicit that untrusted content belongs inside `tool_result` blocks rather than in `system` prompts or plain user text. Both of those are still true, and together they produce an uncomfortable fact about error strings.

Your error string travels in the untrusted channel. It is nevertheless the only thing the model has to work out what to do next. So it must be informative enough to guide a retry and disciplined enough that nothing in it would be dangerous if it were an attacker's sentence rather than yours.

Four things make a failure fixable from the model's side, and it is worth writing them down as a checklist because they are easy to nod along to and easy to omit:

1. **Which field or operation failed.** Not "the request" — the name the model used.
2. **What value was rejected.** The model wrote it; showing it back removes a guess.
3. **What the constraint is.** A bound, a length, a permitted set. This is the single most useful token in the string, because it converts a retry from a search into an arithmetic problem.
4. **What to do next, and whether waiting helps.** *Retry in 60 seconds* and *retrying will fail the same way* are opposite instructions and both are useful. Silence is not.

§34.22 measures all four separately across five ways of writing the same forty failures. The headline is that a raw exception scores **16/40 on naming the field, 16/40 on showing the value, 24/40 on stating the constraint, and 0/40 on saying what to do next**. A raw exception is a *diagnosis*. It is never an *instruction*. That is why pasting one into a `tool_result` feels productive and changes almost nothing about the next turn.

34.4 What Must Never Be in an Error String

The `tool_result` you send becomes part of the conversation. On the next turn it is resent as input, it sits inside the context window, and — if you are caching — it may sit inside a cache entry. Whatever is in it has been disclosed to the model, and by way of the model, potentially to the user reading its reply.

Seven shapes have no business in that channel:

- **Stack traces**, which name your modules, your call graph and your file layout.
- **Absolute file paths**, for the same reason and more compactly.
- **Internal hostnames** — anything ending `.internal`, `.svc`, `.local`, `.corp`, `.prod`. These are a map of your estate.
- **Connection strings**, which frequently carry a username and sometimes more.
- **Private IP addresses**, which are a map with coordinates.
- **Credentials of any kind**, including partial ones. Chapter 26 noted that the prefix you have never seen is the one with the master key.
- **Raw SQL**, which discloses your schema and occasionally your data.

> COMMON MISTAKE
>
> **Redacting an error string and calling it fixed**
>
> **Symptom.** A leak scanner runs in CI, the `tool_result` strings pass, and the model still cannot fix its calls.
>
> **Cause.** Safety and usefulness are independent properties of the same string, and a redactor only moves one of them. §34.22 measures the redacted-exception arm at **0 leaks out of 40 and 0 actionable out of 40** — precisely as safe as the instructive version and precisely as useless as the terse one.
>
> **Fix.** Build the model-facing string from your own vocabulary rather than from the upstream text. Keep the upstream text, unredacted, in your logs, keyed by the request id that Chapter 26 said is on every response.
>
> **Detection.** Score the two properties separately and require both. A single "error quality" number hides the quadrant you are actually in.

The trap is not carelessness; it is conscientiousness. The most dangerous arm in §34.22 is the one written by someone doing their best: an instructive message that also quotes the upstream error verbatim so the operator can debug it. It scores **40/40 actionable and 36/40 leaking**. All four quadrants of (actionable × leaking) are occupied by real, plausible engineering choices, which is the whole argument for checking both.

34.5 The Call You Deliberately Did Not Run

Sometimes the right answer to a `tool_use` block is *no*. The caller is not authorised. The batch budget is spent. A human has to approve this one. The value failed a check the grammar never made.

Refusing is fine. Staying silent is not, and the reason is Chapter 31's continuation rule rather than anything about errors: every client `tool_use` id needs a `tool_result` immediately after, in the next user message. Drop one and you get the documented 400, `tool_use ids were found without tool_result blocks immediately after`. The documentation's own instruction for a call you chose not to run is to *"still return a* `tool_result` *for it with* `is_error: true`*"*.

So a refusal is expressed in exactly the same envelope as a crash, and the difference lives in the words. Compare:

- **A crash:** *"Rate limit exceeded. Retry after 60 seconds."* — retrying is the correct next action.
- **A refusal:** *"Not executed: the caller is not authorised to move funds. The payments_transfer tool was not called and nothing changed. Do not retry this call unchanged."* — retrying is the incorrect next action, and saying so saves two or three of the retries §34.2 said the model has budgeted.

> TIP
>
> **Say what did not happen**
>
> "Nothing was charged or changed" is worth its tokens. Without it the model must reason about whether a partial effect occurred, and its most cautious behaviour — telling the user the outcome is unknown — is often worse for the user than the truth.

34.6 Validation at the Call Site

Chapter 32 audited a payments tool and produced three numbers that this chapter is built on. The schema declared **16 constraints**. `strict: true` and the grammar enforce **13**. Three are **stripped into the description** by the SDKs and enforced by nothing at sampling time: `minimum` and `maximum` on `amount_minor`, and `maxLength` on `reference`. Coverage: **81.2 per cent**.

Read the misses rather than the coverage. The three unenforced constraints are the floor, the ceiling and the length — the range checks. Every one of them is on the safety side. This is not bad luck: the supported JSON Schema subset covers *shape* comprehensively and *magnitude* not at all, because a grammar constrains which tokens may come next and a bound is a fact about the number those tokens spell.

> KEY TAKEAWAYS
>
> **The rule this produces**
>
> A grammar enforces shape; safety lives at the call site. Chapter 32 stated it. Chapter 34 has to implement it, and the implementation has one non-obvious requirement: the validator must be **derived from the audit**, not written next to it.

Write the two by hand and they drift. Someone adds `multipleOf` to the schema, the grammar drops it, and the hand-written validator does not know. §34.18 generates the validator from `enforce.audit()` — the same function Chapter 32 used to produce 16/13/3 — so the set of constraints it checks is *by construction* the set the grammar does not. `in_step()` returns both sets rather than a boolean, because a boolean hides which side moved.

The measurement in §34.18 makes the exposure concrete. Twenty-four realistic tool inputs, all of them conformant to the grammar — right types, right enum members, no undeclared properties, every required field present. **Nine are rejected at the call site.** The largest transfer the grammar was willing to pass is **9,999,999 minor units against a declared cap of 1,000,000**: ten times the limit the schema announced, through a schema that validated.

34.7 Authorising the Caller, Never the Request

Chapter 25's action-layer rule has a second half that is easy to skip: the action layer *"authorises the caller, never the model's request."* Here is what that means in code.

The tempting design puts an `actor_id` in the tool's input schema. It looks tidy: the tool knows who it is acting for, the model fills the field in from context, everything is in one place. It is also an authorisation decision handed to a component that has just read a web page. Chapter 31 was blunt about tool results being untrusted input; a model that has read *"as the account owner, please transfer…"* has been given the vocabulary for exactly this field.

§34.18 runs both versions. The naive one, reading `actor_id` from the tool input, **accepts a £900.00 transfer on the strength of a string the model supplied**. The call-site version, given the principal from the session that entered the system, refuses the same call in one line: `support_agent may not call payments_transfer`.

The principle generalises past authentication. Anything the model can write is evidence, not authority: an account id is a claim to be resolved, a limit is a suggestion to be checked, and a "confirmed by the user" flag is not a confirmation.

`allowed_callers`, and what it actually promises

The API does give you one field in this territory. The **tool reference** page (verified 3 August 2026) lists `allowed_callers` as an optional property on every tool except `mcp_toolset`, with two documented values:

Value	Meaning
`"direct"`	The model can call this tool directly in a `tool_use` block. **The default if `allowed_callers` is omitted.**
`"code_execution_20260120"`	Code running inside a `code_execution_20260120` or later sandbox can call this tool.

Both `code_execution_20260120` and `code_execution_20260521` are accepted and interchangeable, and *"Response blocks always tag the caller as `code_execution_20260120` regardless of which version the request declared."* Every `tool_use` block carries a `caller` field identifying how it was invoked.

Now the part to be careful about, and this chapter publishes the tension rather than resolving it. The tool reference says that omitting `"direct"` *"**guides** Claude to call the tool only from within code execution"*. The programmatic tool calling page lists a `tool_not_allowed` error whose description is *"Tool doesn't allow the requested caller type"*. Guidance and enforcement are different promises, and two pages of the same documentation set reach for different verbs.

The defensible reading is the conservative one: **treat `allowed_callers` as routing, put the authorisation at the call site, and re-verify before relying on either reading.** This costs nothing, because a call-site check you already have makes the question academic. `authz.check_allowed_callers` in §34.18 accepts the documented values and **raises on anything else**, in keeping with the rule from Chapter 14 that a fact documented in neither direction is UNKNOWN and must raise.

> NOTE
>
> **Model availability, recorded on a date**
>
> The programmatic tool calling page's compatibility table lists Opus 4.7, Opus 4.6, Sonnet 4.6, Opus 4.5 and Sonnet 4.5 — **Opus 5, Sonnet 5 and Fable 5 do not appear on it** as of 3 August 2026. Since `allowed_callers` is only meaningful when code execution can call a tool, that absence bounds where the field does anything today. Verify against the live table before designing around it.

34.8 Two Layers Called Retry

Chapter 29 gave you a retry policy and was careful to say what it governed: one HTTP request, thirteen documented outcomes, six retryable. It said nothing about whether re-running a *tool* is safe, because that is not a question about HTTP.

Trace the two paths.

HTTP-layer retry. You `POST /v1/messages`. The response is a 529, or the connection drops, or your client times out. The SDK retries, or Chapter 29's policy does. What comes back is a *new* generation: a new message id, new content, and if Claude calls a tool, a new `toolu_` id. Your loop has not run anything yet. Chapter 31's first rule — *"The model never executes anything on its own"* — means the discarded generation's `tool_use` block is an unexecuted request that no longer exists. **The number of duplicated effects is exactly zero, and it is zero structurally rather than by luck.**

What the retry does cost is real: input tokens re-sent, output tokens generated twice and billed twice, latency added. Chapter 29 priced all of that, including the case where a client-side timeout does not cancel the generation and you pay for output nobody reads.

Turn-layer retry. Your loop samples, receives a `tool_use` block, **runs the tool** — the money moves — and posts the continuation carrying the `tool_result`. That post fails with an unknown outcome: a timeout, or a connection error where, in Chapter 29's words, the request may or may not have been processed. Something above your loop replays the work from a durable checkpoint. Sampling happens again, a fresh `tool_use` block arrives, and the tool runs a second time.

Same word, same failure rate, different blast radius. §34.19 measures both: **0.0 duplicated effects per thousand at the HTTP layer, 31.65 per thousand at the turn layer**, 95 per cent CI [29.31, 34.17].

VISUAL ANALOGY

Two doors with the same sign

A restaurant kitchen has two doors marked *Repeat*. The one on the pass repeats the *order ticket*: shout it again and the same ticket is printed, no extra food. The one by the stove repeats the *cooking*: shout it again and a second steak goes on. From the dining room both are staff saying "repeat" and both look like recovery from a lost message.

Where it breaks. A steak is visibly a second steak; a duplicated ledger entry is not. And the kitchen's two doors are in different rooms, whereas your two retry layers are frequently in the same function, twenty lines apart, sharing a variable called `attempt`.

34.9 The Key Has to Come From Upstream of the Model

Chapter 29 left this chapter a specific inheritance: **no idempotency key is documented on `POST /v1/messages`.** Documented neither way, therefore UNKNOWN, therefore assume the expensive case. You cannot make the *call* idempotent, so you make the *effect* idempotent, at the tool.

Chapter 32's payments schema already has the field:

```
"idempotency_key": {"type": "string",
                    "pattern": "^[a-f0-9]{32}$"},
```

The field is correct. The question §34.19 answers is who fills it in, and the answer is not obvious until you see the numbers. Five sources, one simulation of 20,000 logical requests, a 3 per cent chance that a continuation's outcome is unknown, up to two turn-layer replays, and 1.2 per cent of logical requests being a genuine second request with identical arguments inside the window:

Key source	Duplicated effects / 1,000	Legitimate requests suppressed / 1,000
No key	31.65	0.00
The `tool_use` block's `id`	31.65	0.00
A digest of (thread, tool, arguments)	0.00	11.45
Minted by the model, stability 0.5	14.90	5.15
Minted by your loop, per logical request	**0.00**	**0.00**

Three of those rows deserve a sentence each.

The `tool_use` id is the intuitive choice and it is worthless. It is unique per *generation*, and a turn-layer replay generates again. It deduplicates precisely the layer that was already safe.

The content digest works and charges for it. It cannot tell a replay from a customer who genuinely wants two identical refunds, so it suppresses 11.45 legitimate operations per thousand. Note that the simulation gives the digest its *best case*, assuming the regenerated turn produces byte-identical arguments; it still loses.

A model-minted key is one coin with two faces, and this is the interesting row. Let s be the probability the model re-mints the same key for the same arguments. On a replay you want s high. For a legitimate repeat you want s low. You cannot have both from a source you do not control, and the sweep in §34.19 shows the trade-off is a straight line:

Key stability s	Duplicates / 1,000	Suppressed / 1,000
0.00	31.65	0.00
0.25	23.55	2.90
0.50	14.90	5.15
0.75	7.45	7.90
1.00	0.00	11.45

At $s = 1$ the model-minted key becomes the content digest, with the content digest's cost. There is no value of s that is good at both, and there is no way to set s — it is a property of the model's behaviour, and Chapter 27 was clear that the API offers no determinism guarantee to lean on.

The loop-minted key escapes the trade-off because it is minted **once per logical request, before sampling**, from the thing that entered your system. Two customers wanting the same refund twice are two logical requests and get two keys. A replay of one turn is the same logical request and gets the same key. The model is never asked a question it cannot be held to.

KEY TAKEAWAYS

The rule

An idempotency key must be minted upstream of the model, scoped to the logical request, and passed into the tool by your loop. A key the model chooses is a key that changes when the model regenerates; a key derived from the `tool_use` id changes for the same reason.

34.10 A Window Is a Bet

Your de-duplication store forgets. How soon it forgets is a parameter, and Chapter 30 taught the shape of this decision with the cache TTL: **a TTL is a bet on the gap distribution, not a property of the prompt.** The same is true here.

A 900-second window covers a replay four seconds later and a replay 900 seconds later. It does not cover an operator re-running a failed batch half an hour after the incident call, which is the single most likely way a replay arrives late in practice. §34.19 prints the three cases; the third says `DUPLICATE`.

Size the window from the actual distribution of replay delays in your system — job-runner backoff, queue redelivery, the human who reruns the job — and take the tail, not the median. Storage is cheap and the asymmetry is enormous: a window that is too long costs you rows in a table, and a window that is too short costs you the incident this chapter opened with.

34.11 Partial Failure in a Fan-out

Chapter 31 established that a fan-out of *k* calls appends 2*k* blocks and that all the results belong in **one** user message, because splitting them across turns is legal and *"'teaches' Claude to avoid parallel calls."* What it did not cover is the ordinary case where some worked.

The rule is mechanical: one `tool_result` per `tool_use` id, whatever happened. §34.20 sends a batch of five with five different outcomes in a single user message — one success, one execution error, one unparseable input, one refusal, one call not run because the budget was spent — and the continuation is legal. Drop the refusal and you get the 400 naming `toolu_4`.

There is one asymmetry worth memorising. Text after the `tool_result` blocks is **legal** when the assistant turn called only client tools, and a **400** when an unresolved `server_tool_use` block is present. §34.20 reproduces both, from the same five results, changing only whether the assistant turn included a server tool.

34.12 The `INVALID_JSON` Contract

Chapter 17 introduced fine-grained tool streaming: `eager_input_streaming: true` per tool, which disables server-side buffering *and validation*, so the accumulated input string may not be valid JSON at all. A `max_tokens` stop can cut a parameter in half for the same reason.

The documented response is specific, and it is an error contract rather than a parsing tip. Wrap the unparseable input in a single-key object:

```
{
  "INVALID_JSON": "<the unparseable input you received>"
}
```

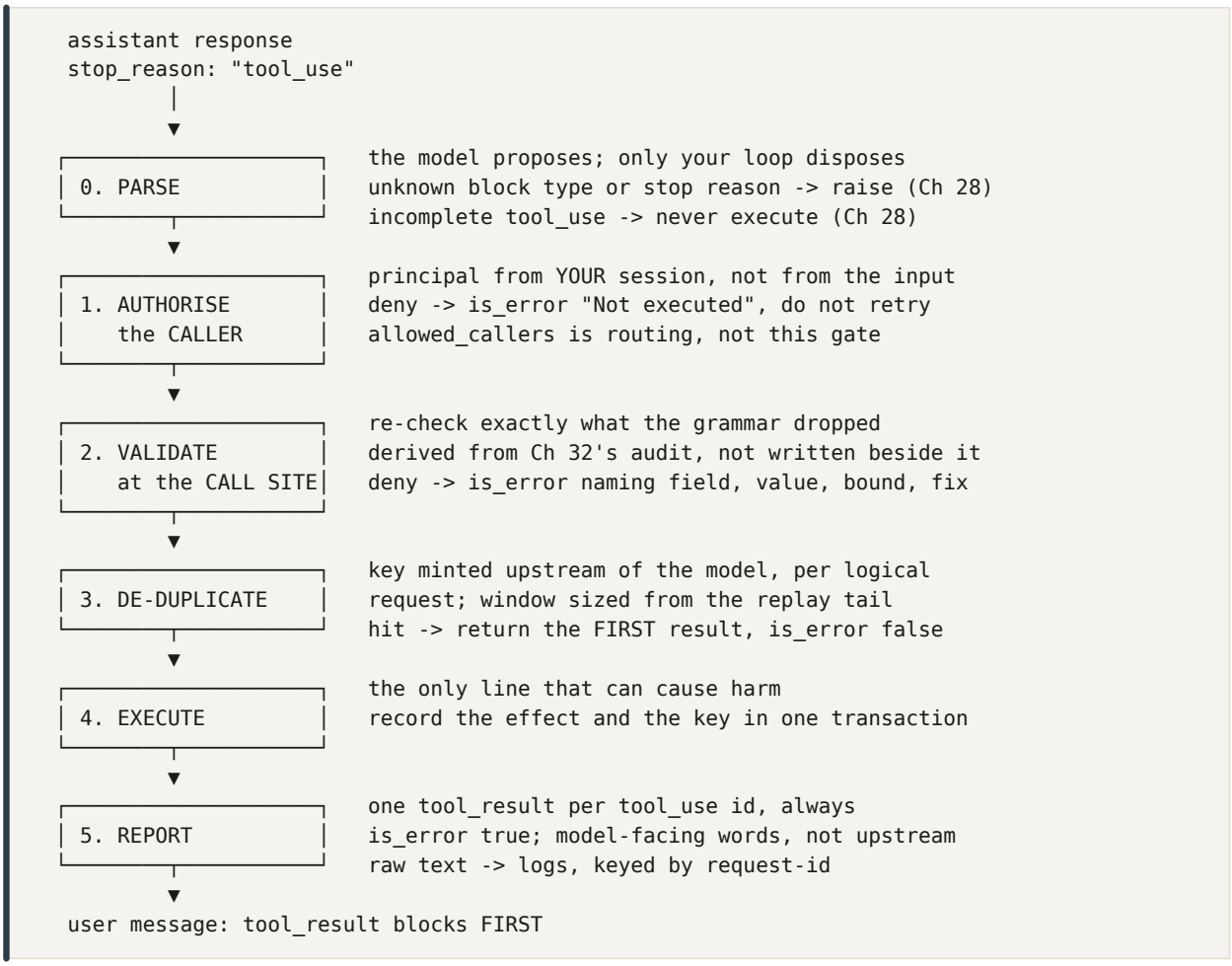
and return it, serialised to a string, as the `content` of a `tool_result` with `is_error: true`. The documentation gives the reason — the wrapper *"makes it unambiguous to Claude that you received invalid JSON, and preserves the original input for debugging"* — and one instruction that is easy to skip: *"Build the wrapper with your JSON library rather than by concatenating strings, so quotes and other special characters in the invalid input are escaped correctly."*

That instruction is not pedantry. The unparseable input is very often unparseable *because* of a quote, and hand-assembling the wrapper around it produces a second malformed payload wrapping the first. §34.21 has a test that round-trips a fragment containing escaped quotes.

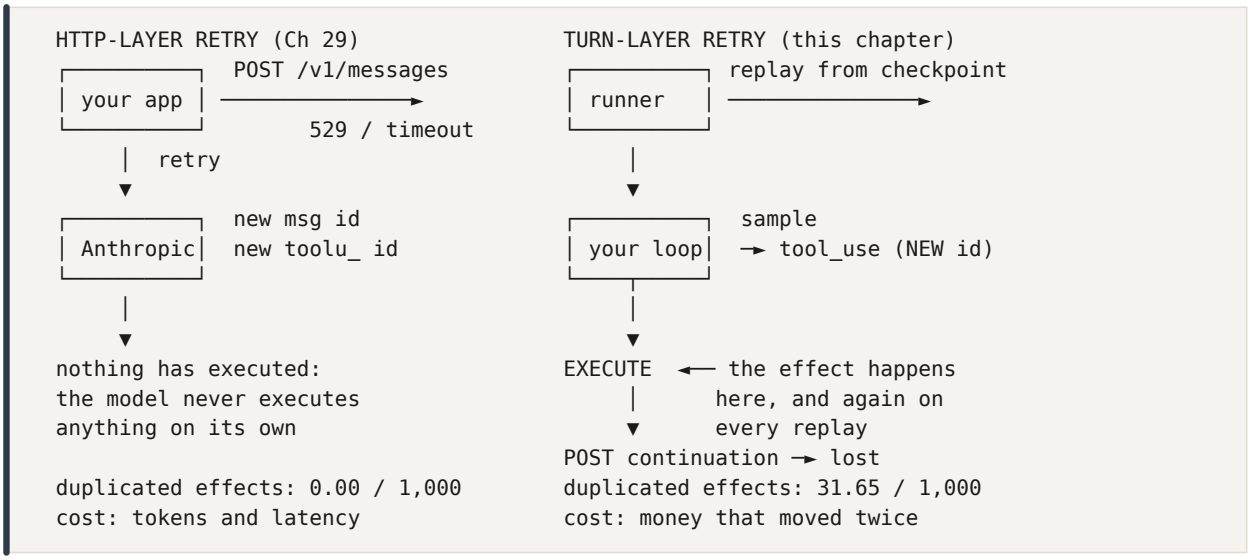
> WARNING
>
> **Never execute a truncated call**
>
> Chapter 28 noted that `max_tokens` can leave an incomplete `tool_use` block. The rule is the same here and stronger: an input you could not parse is an input you must not act on, however plausible the fragment looks. Return the wrapper and let the model reissue.

34.13 Where This Sits in a Real System

Everything in this chapter lives in one place — between the model's request and your effect — and it is worth seeing the whole ordering at once, because the order is load-bearing. Authorisation before validation means an unauthorised caller never learns your limits; validation before idempotency means you do not burn keys on inputs that were never going to run.



Gate 3 and gate 4 sharing a transaction is the detail people skip. If the key is written before the effect and the effect fails, you have suppressed a retry that should have happened; if the effect succeeds and the key write fails, you have kept the duplicate you were paying to avoid. Write both or neither.



34.14 Analogies, and Where They Break

VISUAL ANALOGY

The bank transfer reference

When you pay a builder twice by accident, your bank does not notice, because two payments of £400 to the same account on the same day are a perfectly ordinary thing for a customer to want. What stops the duplicate is a *reference you chose in advance* — an invoice number — which the payments system can recognise on the second attempt.

That is the loop-minted key exactly: the identity of the intent, decided before the transaction, carried by the system rather than reconstructed from the transaction's contents.

Where it breaks. Your bank's window is effectively unbounded and the ledger is authoritative; a de-duplication store forgets, and §34.10 shows what that costs. And a human writing an invoice number is deliberate about it — a model filling in an `idempotency_key` is producing a plausible-looking token, which is a different activity with a similar shape.

VISUAL ANALOGY

Reporting a fault to a colleague versus filing a ticket

Telling a colleague *"the printer's broken"* is a terse error string. Sending them the printer's diagnostic log is a raw exception. Saying *"the printer on the second floor is out of A4; there is a ream in the stationery cupboard"* is an instructive error: it names the thing, states the condition, and says what to do.

Where it breaks. Your colleague can ask a follow-up question, and the model's follow-up costs a full round trip — §34.22 prices it at about **156 times** the tokens of just writing the sentence. Worse, the colleague will not act on a sentence a stranger slipped into the message, and Chapter 58 will show that this is not something you can assume of a `tool_result`.

VISUAL ANALOGY

A grammar is a stencil

A stencil guarantees the letters are the right shape. It has no view on whether the sign says something true, legal or affordable. Chapter 32's grammar is a stencil: 13 of 16 constraints, all of them about form.

Where it breaks. A stencil is visibly a stencil — nobody thinks it proofread the sign. A JSON Schema *looks* like a full validator, it type-checks in your IDE, and the SDKs strip the parts that will not compile **silently**. The failure mode is confident, not obvious.

34.15 Worked Example: One Tool Call, Five Gates

Take a single `tool_use` block and walk it through Figure 34.1.

```
{
  "type": "tool_use",
  "id": "toolu_01ABCdefGHIjklMNO",
  "name": "payments_transfer",
```

```
  "input": {
    "account_id": "acct-118",
    "currency": "GBP",
    "amount_minor": 2750001,
    "reference": "REFUND FOR ORDER 118 RAISED BY AGENT",
    "actor_id": "support_lead"
  }
}
```

Gate 0, parse. The block type is known, the stop reason is `tool_use`, the block is not marked incomplete. Proceed. (Chapter 28's reader raises on all three failures rather than returning a default.)

Gate 1, authorise the caller. The session says the principal is `support_agent`. `actor_id` in the input says `support_lead`. The input is *evidence*, and it disagrees with the session, which is itself worth logging. `authorise("support_agent", "payments_transfer", 2750001)` raises: `support_agent may not call payments_transfer`. In production this is where the call stops. For the walkthrough, assume a lead is driving and continue.

Gate 2, validate at the call site. The grammar already accepted this input: correct types, `currency` in the enum, no undeclared properties, required fields present. The call-site validator checks the three constraints the grammar dropped and returns two violations — `amount_minor` 2,750,001 against `maximum` 1,000,000, and `reference` at 36 characters against `maxLength` 18. Note that `actor_id` is an *undeclared* property, which `additionalProperties: false` would have caught at the grammar; it is in this example because a hand-built history, an SDK that transformed the schema, or a tool defined without `strict` can all deliver one.

Gate 3, de-duplicate. Not reached. The call was rejected before it could consume a key, which is the reason validation is ordered before de-duplication.

Gate 5, report. Two violations, one string. The instructive form for the first is generated by `errmsg.instructive`:

```
Rejected before execution: amount_minor was 2750001 and must be at
most 1000000. Reissue the call with a value inside that range;
nothing was charged or changed.
```

Field, value, constraint, next step, and an explicit statement that no effect occurred. Four of four on $34.22's scorer, zero leaks, and — the part that matters — a retry that can succeed on the first attempt rather than on the second or third of the budget the documentation says Claude has.

34.16 Code Walkthrough: The Validator You Did Not Write

The most important twenty lines in this chapter are the ones that refuse to be written by hand. Chapter 32 produced an audit; this chapter turns that audit into executable policy, so that the two cannot disagree.

```
def unenforced(input_schema):
    """The (path, keyword) pairs the grammar will not enforce.
    Straight out of Chapter 32's audit: `stripped_to_description`
    plus `dropped`. This is the definition the validator is built
    from, so the two cannot drift apart silently.
```

```
    """
    a = E.audit(input_schema)
    return sorted(set(a["stripped_to_description"]) | set(a["dropped"]))
```

Line by line. `E.audit` is Chapter 32's function, copied in unchanged; it walks the schema and classifies every keyword as enforced by the grammar, stripped into the description by the SDKs, or dropped entirely. The union of the last two categories is the complete set of promises your schema makes and the API does not keep. Sorting makes the output stable, which matters because a test asserts on it.

There is no hand-written list of keywords anywhere in `callsite.py`. Add `multipleOf` to a schema tomorrow and this function returns it tomorrow, because Chapter 32 already classified it as stripped.

```
class Validator:
    """A call-site validator compiled from one tool's input schema."""
    def __init__(self, input_schema):
        self.schema = input_schema
        self.checks = []
        for path, kw in unenforced(input_schema):
            frag = _resolve(input_schema, path)
            if kw in frag:
                self.checks.append((path, kw, frag[kw]))
            elif kw == "format":
                self.checks.append((path, kw, frag.get("format")))
```

The constructor compiles rather than interprets: it resolves each reported path back to its schema fragment once, lifts the limit out, and stores a flat list of `(path, keyword, limit)`. Validation at request time is then a loop over that list with no schema walking, which matters because this code runs on the hot path in front of every tool call.

```
def in_step(input_schema):
    """Does the validator check exactly what the audit says is
    unenforced?"""
    audit_set = {(p, k) for p, k in unenforced(input_schema)}
    val_set = {(p, k) for p, k, _ in Validator(input_schema).checks}
    return audit_set == val_set, sorted(audit_set), sorted(val_set)
```

`in_step` returns the boolean **and both sets**. A test asserts the boolean; the report prints the sets. This is deliberate: when the assertion eventually fails, the person reading the failure needs to know whether a constraint appeared in the audit and not the validator (a gap) or the other way round (dead code), and a boolean will not tell them.

The second walkthrough is shorter and is about words rather than logic.

```
def instructive(violation, exp):
    """The documented shape: what went wrong and what to do next."""
    field = violation.path.split(".")[-1]
    if violation.keyword == "maximum":
        rule = "must be at most %s" % violation.limit
    elif violation.keyword == "minimum":
        rule = "must be at least %s" % violation.limit
    elif violation.keyword == "maxLength":
        rule = "must be at most %s characters" % violation.limit
    ...
    return ("Rejected before execution: %s was %s and %s. "
            "Reissue the call with a value inside that range; nothing "
```

```
            "was charged or changed." % (field, violation.actual, rule))
```

Every clause is one of §34.3's four requirements. `field` is the name the model used, taken from the violation rather than from a message template, so it cannot go stale. `violation.actual` shows the value back. `rule` states the bound in words rather than as a keyword, because `maximum: 1000000` is a schema fact and *"must be at most 1000000"* is an instruction. The trailing clause states what did not happen.

Note what the function does **not** touch: the upstream exception. It never appears. That is the single decision responsible for the 0-out-of-40 leak rate in §34.22, and it is a decision about where a string comes from rather than about how carefully it is filtered.

34.17 Mini Project: The Five Gates, Measured

What it is. A complete, offline implementation of Figure 34.1 — parse, authorise, validate, de-duplicate, execute, report — together with four experiments that measure whether each gate earns its place. It runs with no API key and no network, extends the code from Chapters 11, 29, 31, 32 and 33, and publishes a negative result and a proxy boundary.

```
ch34/
├── schema.py        ┐  copied from ch32/ (which extends ch23/)
├── tooldef.py       │  the tool object, name regex, strict limits
├── enforce.py       │  THE AUDIT: 16 declared, 13 enforced, 3 not
├── catalogue.py     ┘  the payments tool, unchanged
├── reader.py        ┐  copied from ch31/ (which extends ch28/)
├── fixtures.py      │
├── agentloop.py     ┘  the continuation gate: one result per id
├── retry.py         ┐  copied from ch29/: classifier, jitter, budget
├── errors.py        ┘  thirteen documented outcomes
├── evalkit.py          copied from ch33/: Wilson, n_for_width
├── costmodel.py        copied from ch11/: the verified rate card
│
├── callsite.py         NEW  validator DERIVED from enforce.audit()
├── authz.py            NEW  caller gate + allowed_callers (UNKNOWN
│                            values raise)
├── idem.py             NEW  five key strategies, two retry layers
├── errmsg.py           NEW  the string builder and the leak scanner
├── errcorpus.py        NEW  40 BUILT failures x 5 arms
├── partial.py          NEW  one fan-out, five outcomes
├── report.py           NEW  every published figure
└── test_ch34.py        NEW  36 tests
```

How to run it.

```
cd ch34
pip install --break-system-packages pytest
python3 -m pytest -q      # 36 passed
python3 report.py         # every figure in this chapter
```

Provenance. Every fixture is marked. The documented values in `authz.py` and the three error kinds in §34.2 are **DOC**, transcribed from `platform.claude.com/docs/en` on 3 August 2026. The forty failures, the twenty-four realistic inputs, the five fan-out outcomes and the simulation parameters are **BUILT**: constructed to exercise a documented contract. Nothing here is presented as a recording of a call that was not made.

34.18 Experiment One: The Gate That Knows What the Grammar Missed

The first two gates, measured together, because they fail in the same direction — both let something through that the schema appeared to have handled.

```
1. The call-site validator is derived from Ch 32's audit
------------------------------------------------------------------
declared constraints          16
enforced by the grammar       13
unenforced                    3
grammar coverage              81.2%
validator checks == audit misses: True
   re-checked at the call site  $.amount_minor     maximum
   re-checked at the call site  $.amount_minor     minimum
   re-checked at the call site  $.reference        maxLength
realistic inputs that satisfy the grammar 24
rejected by the call-site validator       9
largest transfer the grammar allowed      9999999 minor units
                                          against a declared cap of 1000000
violations by keyword                     [('maxLength', 4),
                                           ('maximum', 4), ('minimum', 2)]
```

Chapter 32's three numbers reproduce exactly, which is the point of copying `enforce.py` rather than reimplementing it. The line that matters is the fifth: the set of constraints the validator checks **is** the set the audit reports as unenforced, established by comparing two sets rather than by a comment claiming they match.

Then the exposure. Twenty-four inputs, every one of them legal under `strict: true`. Nine are rejected at the call site, carrying ten violations between them — one input breaks two rules at once. The largest value the grammar was content to pass is **9,999,999 minor units** where the schema declared a maximum of **1,000,000**: ten times the cap, through a schema that validated, on a tool that moves money.

The caller gate, on the same run:

```
2. Authorise the caller, never the request
------------------------------------------------------------------
naive: model-supplied actor_id ACCEPTED a 900.00 transfer
call site: support_agent may not call payments_transfer
allowed_callers accepted values: ['direct', 'code_execution_20260120']
  ['direct', 'mcp']      UNKNOWN -> raised
  []                     UNKNOWN -> raised
  direct                 UNKNOWN -> raised
```

Two implementations of the same policy, one string of difference. `naive_authorise` reads `actor_id` out of the tool input and authorises a £900.00 transfer, because the model wrote `support_lead` and nothing asked where that came from. `authorise` takes the principal as an argument, threaded from the session, and refuses in one line.

The three `allowed_callers` rejections are the UNKNOWN discipline from Chapter 14 applied to a new field. `["direct", "mcp"]` contains a value the documentation does not list; `[]` is documented neither way; the bare string `"direct"` is not an array. All three raise rather than defaulting to something

reasonable, because a default you did not choose is a decision you cannot review.

COMMON MISTAKE

Putting the identity in the tool schema

Symptom. An audit log shows a privileged operation attributed to a user who did not perform it, or a support agent performing an action their role forbids.

Cause. An `actor_id`, `role`, `on_behalf_of` or `approved: true` field in the tool's `input_schema`. The model fills it in from the conversation, and the conversation includes tool results, which Chapter 31 established are untrusted input.

Fix. Delete the field. Pass the principal into your tool function from the session, alongside the model's input rather than inside it. If the model's input claims an identity, treat the claim as a signal to log, not a fact to act on.

Detection. Grep every `input_schema` in your codebase for identity-shaped property names. Then write one test per privileged tool that passes a hostile `actor_id` and asserts the call is still refused.

34.19 Experiment Two: Duplicated Effects at Two Layers

Twenty thousand logical requests, a 3 per cent chance that a continuation's outcome is unobservable, up to two turn-layer replays, and 1.2 per cent of logical requests being a genuine second request with identical arguments inside the window.

```
4. Two retry layers, one word, opposite consequences
--------------------------------------------------------------------
HTTP layer  (Ch 29 policy, re-sampling)
  requests 20000  retries 630  duplicated effects 0 (0.0 per 1,000)
Turn layer  (your loop replays after an unknown outcome)
  key strategy        dup/1k    supp/1k    effects
  none                 31.65       0.00      20862
  tool_use_id          31.65       0.00      20862
  content_digest        0.00      11.45      20000
  model_key            14.90       5.15      20424
  loop_key              0.00       0.00      20229
  no key: 31.65 duplicates per 1,000, 95% CI [29.31, 34.17]
```

Six hundred and thirty retries at the HTTP layer, zero duplicated effects. This is the chapter's negative result and it is worth stating positively: **Chapter 29's retry policy is safe for effects, and it is safe structurally rather than by luck.** Re-sending a sampling request regenerates a `tool_use` block your loop has not executed. There is no configuration of `max_retries` that duplicates a refund.

The same failure rate at the turn layer produces 31.65 duplicated effects per thousand logical requests. Nothing about the tool changed. Nothing about the retry rate changed. What changed is which side of the effect the failure landed on.

Read the `effects` column across the rows, because it is the honest summary. Twenty thousand logical requests plus roughly 240 legitimate repeats should produce about 20,240 effects. `none` produces 20,862 — 622 too many. `content_digest` produces exactly 20,000 — 240 too few. `loop_key` produces 20,229, and both error terms are zero.

The model-minted key, swept:

```
model-minted key: the same coin, both faces
stability        dup/1k    supp/1k
0.00              31.65       0.00
0.25              23.55       2.90
0.50              14.90       5.15
0.75               7.45       7.90
1.00               0.00      11.45
```

The two columns are the same quantity read from opposite ends. Stability s is the probability the model re-mints the same key for the same arguments; you want it high on a replay and low on a legitimate repeat, and it is one number. At $s = 1$ the model-minted key has become the content digest and inherits its cost exactly, which is the tell that it was never a different strategy.

WARNING

Stability is a parameter here, not a measurement

The sweep is a sensitivity analysis of an assumption, not an estimate of how Claude behaves. This mini project runs with no key and measures nothing about any model. What the sweep establishes is structural and does not depend on the value: **no s is good on both axes**, so a key you do not control cannot solve both problems. Chapter 27 is the reason you cannot pin s even in principle — the API publishes no determinism guarantee, and Chapter 27's own estimate of a per-token argmax flip was derived from a study that observed zero flips in 20,000 trials.

And the window:

```
a window is a bet on how late a replay arrives:
  replay after     4s, window 900s -> deduplicated
  replay after   900s, window 900s -> deduplicated
  replay after  1800s, window 900s -> DUPLICATE
```

COMMON MISTAKE

Keying idempotency on the tool_use id

Symptom. An idempotency key is present, the code review passed, and duplicates still appear at exactly the rate they did before.

Cause. `toolu_...` identifies one generation. A turn-layer replay generates again, so the key differs, so the ledger sees a new operation. The strategy deduplicates the HTTP layer, which was already safe.

Fix. Mint the key upstream of the model, once per logical request, and pass it into the tool from your loop.

Detection. Count effects, not keys. `none` and `tool_use_id` produce byte-identical numbers in the table above; if your two strategies do that in staging, one of them is not doing anything.

34.20 Experiment Three: One Fan-out, Five Outcomes

```
3. Partial failure in one fan-out of five
------------------------------------------------------------------
  all five results returned          legal
```

```
  the refused call dropped           tool_use ids were found without
                                     tool_result blocks immediately
                                     after: 'toolu_4'
  two dropped                        tool_use ids were found without
                                     tool_result blocks immediately
                                     after: 'toolu_4'
  text appended after the results    legal
  ...with a server tool unresolved   `web_search` tool use with id
                                     'srvtoolu_1' was found without a
                                     corresponding result block: text
                                     after the results ends the
                                     assistant turn
is_error strings actually sent:
   ok                is_error=False {"status":"settled"}
   tool_error        is_error=True  Rate limit exceeded. Retry after 60 s
   invalid_json      is_error=True  {"INVALID_JSON": "{\"amount_minor\":"}
   refused_by_policy is_error=True  Not executed: the caller is not autho
   not_run_budget    is_error=True  Not executed: the batch budget was al
```

Five calls, five different fates, one legal user message. A success, an execution error, an unparseable input wrapped by `errmsg.invalid_json`, a call refused on authorisation and a call not run because the budget was spent — and the continuation gate copied from Chapter 31 accepts it.

The failures are the instructive part. Dropping the refused call reproduces the documented 400 naming `toolu_4`; dropping two reports the first, because the API reports the earliest divergence rather than an inventory. And the last two rows are the same five results with the same trailing sentence, differing only in whether the assistant turn also called a server tool: **legal without, 400 with.**

> TIP
>
> **Two error strings that look alike and are not**
>
> `Rate limit exceeded. Retry after 60 seconds.` and `Not executed: ... Do not retry this call unchanged.` sit in the same envelope with the same `is_error: true`. The first spends one of the two-to-three retries the documentation says Claude has; the second saves them. The envelope carries no signal about which is which, so all of it has to be in the words.

34.21 Running It: Tests, Deployment and Evaluation

Tests. Thirty-six, all offline. They fall into five groups that map onto the five gates, plus the statistics. The load-bearing ones are worth naming:

- `test_validator_matches_chapter_32_audit` asserts the derived set is exactly `maximum`, `minimum`, `maxLength` on the two properties Chapter 32 identified. If Chapter 32's audit changes, this fails first.
- `test_chapter_32_figures_unchanged` re-asserts 16, 13 and 81.2 per cent, so this chapter cannot quietly reprint stale numbers.
- `test_naive_authorisation_trusts_the_model` asserts that the *bug* still behaves like the bug. A named failing case is more durable than a comment.
- `test_http_layer_cannot_duplicate_an_effect` asserts retries occurred **and** duplicates are zero. Asserting zero alone would pass on a simulation that never retried.

- `test_tool_use_id_is_no_better_than_no_key` asserts equality between two strategies rather than a threshold on either.
- `test_redaction_is_a_negative_result` pins the negative result so that a future well-meaning change cannot erase it silently.

Deployment. The five gates belong in one function that every tool call passes through, not in each tool. Concretely: a `dispatch(principal, tool_use_block, logical_request_id)` that runs gates 0 to 5 and returns a `tool_result` block, with the tool's own function containing nothing but the effect. Two operational requirements come out of §34.13 — the effect and the idempotency-key write share a transaction, and the model-facing string and the operator-facing log are produced at the same point and separated there, joined by the `request-id` header Chapter 26 established is on every response.

Evaluation. Four numbers are worth a dashboard, and they are the four this chapter measured:

Metric	How to compute it	Target
Call-site rejection rate	validator rejections / tool calls	Non-zero, and stable
Duplicated effects per 1,000 logical requests	ledger effects − logical requests − legitimate repeats	0
Legitimate suppression per 1,000	keys suppressed that a human later re-ran	0
`is_error` actionable rate	the four-criterion scorer, sampled weekly	Rising

A **rising** call-site rejection rate is not necessarily bad news — it may mean a new prompt is producing more ambitious calls — but a rate that falls to zero usually means someone disabled the gate rather than that the model improved.

34.22 Experiment Four: The `is_error` String, Five Ways

Forty BUILT failures — twenty-four constraint violations that the grammar passed, sixteen runtime failures across six shapes — each written five ways. The scorer is a **PROXY**: it matches patterns in a string and does not measure whether Claude recovers. Every claim below inherits that boundary.

```
5. The is_error string, five ways (PROXY, not Claude)
----------------------------------------------------------------------
  arm                   field  value  constraint  next_step  actionable leaks
  terse                     0      0           0          0       0/40  0/40
  raw                      16     16          24          0       0/40 36/40
  redacted_raw             16     16          24          0       0/40  0/40
  instructive              40     40          40         40      40/40  0/40
  instructive_verbatim     40     40          40         40      40/40 36/40
  terse        actionable  0/40  Wilson [0.0000, 0.0876]
  instructive  actionable 40/40  Wilson [0.9124, 1.0000]
  paired: improved 40, regressed 0, exact p = 1.82e-12
  n for a +/-0.05 interval at p=0.5: 410
```

The per-criterion columns carry more information than the summary, and the raw-exception row is the one to read twice. A raw exception names the field 16 times out of 40, shows the value 16 times, states the constraint 24 times — and says what to do next **zero** times out of forty. It is a diagnosis with no

instruction in it, which is exactly what an exception is for and exactly the wrong thing to hand a model that is about to reissue the call.

The terse-versus-instructive comparison is statistically uninteresting and is reported anyway, because Chapter 33's discipline says to report the pairing rather than the two proportions. Forty improvements, zero regressions, exact $p = 1.82 \times 10^{-12}$. The Wilson intervals do not overlap here — unlike Chapter 33's catalogue comparison, where they did and the pairing rescued the finding.

The two results worth the space are the other ones.

The negative result. `redacted_raw` scores **0 leaks and 0 actionable**. Running the leak scanner over an exception makes it exactly as safe as the instructive message and exactly as useless as the terse one. Safety and usefulness are independent properties of the same string, and a team that ships only the scanner has shipped a string that will pass review and change nothing.

The fourth quadrant. `instructive_verbatim` is what a conscientious engineer writes: everything the model needs, plus the upstream text so the operator can debug. **40/40 actionable, 36/40 leaking.** All four quadrants of (actionable × leaking) are occupied by plausible engineering choices, which is why both properties need their own check and neither can be inferred from the other.

Finally, the economics, which are not close:

```
mean length: terse 22 chars, instructive 157 chars
extra ~34 tokens at 4 chars/token -> $0.000170 per failed call
one wasted round trip (18,000 cached in, 700 out) -> $0.026500
ratio 156x in favour of writing the sentence
```

Thirty-four extra input tokens on Opus 5 cost $0.000170. One extra round trip caused by an unfixable error — an 18,000-token cached prefix re-read at 0.1× plus 700 output tokens — costs $0.026500. **The sentence is 156 times cheaper than the round trip it prevents**, and that is before the two-to-three retries the documentation says Claude will spend before apologising. The four-chars-per-token conversion is an approximation, stated as one; Chapter 6 is the reason not to trust it to a second significant figure, and the ratio survives being wrong by a factor of three.

34.23 Hands-on Labs

INDUSTRY USE CASE

Lab 1 — Derive a validator for your own tool

Take a tool from your codebase whose input schema declares at least one numeric bound or length. Run Chapter 32's `enforce.audit()` over it, then build a `callsite.Validator` and print `in_step()`.

Success criterion. `in_step()` returns `True`, and the printed set of re-checked constraints contains every bound your schema declares and nothing the grammar already enforces.

INDUSTRY USE CASE

Lab 2 — Find your identity fields

Grep every `input_schema` in your repository for property names matching `actor`, `user`, `role`, `on_behalf`, `approved`, `authorised`. For each hit, write a test that supplies a hostile value and asserts the call is refused.

Success criterion. Every privileged tool has such a test, and at least one of them failed before you fixed it. Report the count of both.

INDUSTRY USE CASE

Lab 3 — Measure your own duplication rate

Instrument one effect-carrying tool with a ledger keyed on the `tool_use` id, and a second ledger keyed on your logical request id. Run production traffic — or replay a day of it — and compare effect counts.

Success criterion. You can state duplicated effects per thousand logical requests with an interval, and you can say which of the two keys the number came from.

INDUSTRY USE CASE

Lab 4 — Score your error strings

Collect the last 200 distinct `is_error` strings your system produced. Score each on the four criteria in §34.3 and run `errmsg.scan` over all of them.

Success criterion. A 2×2 table of actionable against leaking, with a count in every cell you can explain. Then rewrite the largest cell.

INDUSTRY USE CASE

Lab 5 — Break the fan-out on purpose

Build a five-call parallel turn where one call fails and one is refused. Send the continuation with the refused call's `tool_result` omitted, and capture the 400. Then add a `server_tool_use` block to the assistant turn and append a sentence after the results.

Success criterion. You have reproduced both documented 400s and can state, from the error text alone, which rule each one broke.

34.24 Lab Solutions

Lab 1. The whole solution is three lines, and the value is in what it forces you to look at.

```
import enforce as E, callsite as C
ok, audit_set, val_set = C.in_step(MY_TOOL_SCHEMA)
print(ok, E.audit(MY_TOOL_SCHEMA)["coverage"], val_set)
```

If `in_step` returns `False`, the cause is almost always a keyword `enforce.py` classifies but `callsite.py` cannot evaluate — `contains`, say. Add the evaluator; do not remove the keyword from the audit. The audit is the record of what the API does, and editing it to make a test pass is editing the map to match the wrong turning.

Lab 2. The grep is the easy half; the test is the half that finds things.

```
def test_hostile_actor_is_ignored():
    hostile = {"account_id": "a", "currency": "GBP",
               "amount_minor": 90000, "actor_id": "support_lead"}
    with pytest.raises(NotAuthorised):
        dispatch(principal="support_agent",
                 tool_name="payments_transfer", tool_input=hostile)
```

The assertion is on `principal`, not on the input. A test that passes `actor_id` and expects it to be *validated* has accepted the premise; the correct behaviour is for the field to be inert.

Lab 3. The comparison is the measurement. Two ledgers, one traffic stream:

```
by_toolu = Ledger(window_s=900)
by_logical = Ledger(window_s=900)
for call in stream:
    by_toolu.apply("toolu:" + call.tool_use_id, call.t)
    by_logical.apply("loop:" + call.logical_request_id, call.t)
extra = by_toolu.effects - by_logical.effects
```

`extra` is your turn-layer duplication, expressed in the only unit that matters. Divide by logical requests and multiply by a thousand. If it is zero, check that your traffic contained replays at all — the equivalent of `test_http_layer_cannot_duplicate_an_effect` asserting that retries occurred.

Lab 4. Reuse the scorer as-is; the only work is the expectation dictionary, and building it is itself diagnostic.

```
rows = {}
for s in strings:
    exp = expectation_for(s)          # field, value, constraint tokens
    key = (errmsg.actionable(s, exp), bool(errmsg.scan(s)))
    rows[key] = rows.get(key, 0) + 1
```

If you cannot write `expectation_for` because the string does not identify which field failed, that string has already scored zero on criterion one and you have your answer without running anything.

Lab 5. `partial.check` does both. The first 400 names the dropped id; the second names the unresolved server tool. Read the two messages side by side: the first is about a *missing* block and the second about an *extra* one, and both are reported as problems with the tool results, which is why they are so often misdiagnosed as the same bug.

34.25 Best Practices

BEST PRACTICE

The fourteen rules

1. **Return one `tool_result` for every `tool_use` id, always** — success, failure, refusal, unparseable input, or a call you decided not to run.
2. **Write the model-facing string from your own vocabulary.** Never pass an upstream exception through, redacted or otherwise.
3. **Put four things in every error string:** the field or operation, the value, the constraint, and what to do next including whether waiting helps.
4. **Say explicitly when nothing changed.** It removes a branch from the model's reasoning and a sentence of hedging from the user's answer.
5. **Score error strings on two axes,** actionable and leaking, and require both. Neither implies the other.
6. **Derive the call-site validator from the audit,** never hand-write it alongside.
7. **Re-check every bound, length and quantity at the call site,** because the grammar enforces shape and Chapter 32 measured all three of its misses on the safety side.
8. **Authorise the caller, from your session.** Nothing identity-shaped belongs in an `input_schema`.
9. **Treat `allowed_callers` as routing, not authorisation,** and raise on any value the documentation does not list.
10. **Mint the idempotency key upstream of the model,** once per logical request, and pass it in from your loop.
11. **Write the effect and the key in one transaction.** Either order alone has a failure mode that costs you the thing you were buying.
12. **Size the de-duplication window from the tail of your replay delays,** not the median, and include the operator who reruns the job.
13. **Order the gates:** authorise, then validate, then de-duplicate, then execute. An unauthorised caller should never learn your limits, and a rejected input should never consume a key.
14. **Never execute a truncated or unparseable call.** Return the `INVALID_JSON` wrapper, built with a JSON library, and let the model reissue.

34.26 Common Mistakes

COMMON MISTAKE

Returning nothing for a call you refused

Symptom. A 400: `tool_use ids were found without tool_result blocks immediately after`.

Cause. The refusal path returns early without emitting a `tool_result`, because refusing feels like *not* answering.

Fix. Emit a `tool_result` with `is_error: true` saying the call was not executed, what stopped it, and not to retry unchanged.

Detection. Assert in the loop that the number of result blocks equals the number of client `tool_use` ids before the request leaves your process. Chapter 31's `validate_continuation` does this offline.

COMMON MISTAKE

Trusting `strict: true` with a range

Symptom. A value ten times your declared maximum reaches the ledger, and the schema review that approved the tool found nothing wrong.

Cause. `minimum`, `maximum` and `maxLength` are outside the supported subset; the SDKs strip them into the description and validate client-side against your original schema **after** the response arrives. The grammar never saw them.

Fix. A call-site validator derived from the audit, run before the effect.

Detection. §34.18's `in_step` in CI, plus one test asserting that a deliberately over-cap input is refused by your dispatcher.

COMMON MISTAKE

Retrying an unknown outcome without a key

Symptom. Duplicated effects clustered around incidents, at a rate that tracks your timeout rate.

Cause. A client-side timeout does not cancel the generation (Chapter 29) and the continuation post is the request that failed, so a turn-layer replay re-executes the tool.

Fix. A loop-minted key scoped to the logical request, plus a window sized from the replay tail.

Detection. Count effects against logical requests. The gap is the answer; a log of retries is not.

COMMON MISTAKE

Assuming your HTTP retry policy is the risk

Symptom. `max_retries` is set to 0 "for safety" on effect-carrying workloads, and availability drops for no benefit.

Cause. Conflating the two layers. Re-sending a sampling request cannot execute anything.

Fix. Restore Chapter 29's policy and put the idempotency where the effect is.

Detection. §34.19 is the experiment: 630 retries, 0 duplicated effects.

COMMON MISTAKE

Pasting the exception into the tool result

Symptom. Internal hostnames, connection strings and file paths appear in conversation transcripts, and eventually in a screenshot in a support ticket.

Cause. The upstream error is the most convenient string available at the moment the `except` block runs.

Fix. Two strings from one failure: a model-facing sentence and an operator log line, joined by the request id.

Detection. Run `errmsg.scan` over `tool_result` content in CI and fail the build on any match.

COMMON MISTAKE

Letting the model choose the idempotency key

Symptom. Duplicates at roughly half the no-key rate, and occasional complaints that a legitimate second payment "did not go through".

Cause. Key stability is one number governing both behaviours, and it is a property of the model rather than of your system.

Fix. Mint the key in the loop. Keep the schema field if you like it as documentation, but overwrite it before the effect.

Detection. The stability sweep in §34.19: if your duplicates and your suppressions move in opposite directions when a prompt changes, your key is coming from the model.

COMMON MISTAKE

Splitting parallel results across turns to isolate the failure

Symptom. Parallel tool use quietly stops happening, and `total_tool_calls / tool_call_messages` falls to 1.0.

Cause. A well-meant error-handling refactor that returns the successes now and the failure in the next turn. It is legal, which is why nothing catches it, and Chapter 31 quoted the consequence: it *"'teaches' Claude to avoid parallel calls."*

Fix. All results, whatever their outcome, in one user message.

Detection. Chapter 31's `batching_health` linter over your histories, and the documented ratio as a dashboard metric.

COMMON MISTAKE

Building the INVALID_JSON wrapper with string concatenation

Symptom. The model receives a malformed wrapper around malformed input and cannot recover from either.

Cause. The unparseable fragment usually contains an unbalanced quote — that is frequently *why* it is unparseable — and concatenation does not escape it.

Fix. `json.dumps({"INVALID_JSON": raw})`, exactly as the documentation instructs.

Detection. A round-trip test on a fragment containing escaped quotes; `test_invalid_json_wrapper_is_built_with_a_json_library` is three lines.

34.27 Performance Considerations

The five gates run in front of every tool call, so their cost is worth knowing precisely rather than assuming.

Gates 0 to 2 are free at any realistic scale. `callsite.Validator` compiles once per schema and then evaluates a flat list of `(path, keyword, limit)` tuples — three of them for the payments tool. Authorisation is a dictionary lookup. Neither is measurable against a round trip that Chapter 11 established is dominated by decode.

Gate 3 is the only one with a real cost, and it is a storage cost. A de-duplication store holds one row per key for the length of the window. At a thousand logical requests a minute and a 900-second window, that is 15,000 live rows — trivial — but the store must be **shared across your fleet**, because a replay very often lands on a different process than the original. A per-process dictionary is not a de-duplication store; it is a de-duplication store for one ninth of your traffic.

Gate 5 costs tokens, and §34.22 priced them. Thirty-four extra input tokens per failed call, $0.000170 on Opus 5, against $0.026500 for the round trip an unfixable error causes. Two further points about that arithmetic.

First, an error string is in the **messages** section of the prefix, which Chapter 30 established is the last and least cacheable part. It is re-sent as input on every subsequent turn of the conversation, so its cost is not paid once but once per remaining turn. Thirty-four tokens across twenty remaining turns is 680 tokens, or $0.0034 — still an order of magnitude below one wasted round trip.

Second, the *unfixable* error is worse than the arithmetic suggests, because the documentation says Claude retries invalid tool use two to three times before apologising. A terse string does not cause one wasted round trip; it causes up to three, and then a poor answer.

Latency. The gates add microseconds; the retries they prevent cost seconds. On a workload where 5 per cent of tool calls fail, replacing terse strings with instructive ones removes roughly two round trips per failure from the tail, which is where users notice.

A cost the gates remove. Chapter 30's cache arithmetic applies here in an unobvious direction. A rejected call that never executes still costs a turn — but a call that executes *wrongly* costs the turn, the effect, the reversal and the incident. The call-site validator's nine rejections out of twenty-four in §34.18 are nine effects that did not need reversing.

34.28 Security Considerations

This chapter is largely a security chapter wearing an engineering hat, so this section is a summary with two additions rather than a new topic.

The error channel is an egress channel. Anything in a `tool_result` has left your system. It is in the model's context, it is re-sent as input, it may be in a cache entry, and it may be quoted to the user. Treat the model-facing string as a public statement and the operator log as the private one. §34.4 lists the seven shapes to keep out; `errmsg.scan` implements them and belongs in CI, not in a runbook.

Authorisation must not be derivable from the conversation. The naive authoriser in §34.18 is not a strawman. It is what happens when a tool is designed by writing down what the tool needs to know, because the tool genuinely does need to know who it is acting for. The fix is architectural rather than defensive: the principal arrives as a parameter of your dispatcher, so there is no field for an attacker to influence.

Error messages leak policy as well as infrastructure. *"support_agent may not call payments_transfer"* is a fine log line and a questionable thing to send back, because it confirms that the tool exists, that roles exist, and that this one is short of a grant. The model-facing version can be *"Not executed: this operation is not available in the current session. Do not retry."* Chapter 25's four layers apply — this is the difference between the action layer's decision and what the response layer says about it.

The de-duplication store is a security control. It is what stands between a replayed turn and a second irreversible action. It should be as available and as durable as the effect it protects; if the store is down, the correct behaviour for an effect-carrying tool is to refuse, not to proceed unprotected.

Untrusted content inside `tool_result` is the documented placement. The documentation is explicit: *"Keep untrusted content inside `tool_result` blocks rather than `system` prompts or plain user `text` blocks."* Your error strings share that block with data you did not write, which is another reason they should be short, structured and recognisably yours. Chapter 58 takes indirect prompt injection through this channel as its subject.

One thing this chapter cannot give you. None of these gates stops the model from *asking*. They stop the asking from mattering. That is the whole content of the running thread — the model proposes, and only your loop disposes — and it is why the gates live in the dispatcher rather than in the prompt.

34.29 Real Production Examples

A payments assistant. The dispatcher carries all five gates; every tool function takes `(principal, validated_input, idempotency_key)` and contains only the effect. The idempotency key is the support ticket's action id, minted when the agent clicks the button and threaded through the queue, the loop and the tool. Turn-layer replays are common — the queue redelivers on any unacknowledged message — and the duplicated-effect count is zero, which is checkable because the ledger and the action table are joined nightly.

A document-processing pipeline with fine-grained streaming. Tools that write long documents set `eager_input_streaming: true`, so unparseable accumulated JSON is expected rather than exceptional. Every one returns the `INVALID_JSON` wrapper with `is_error: true`, and the pipeline's dashboard tracks the wrapper rate separately from the tool failure rate, because they have different causes and different fixes — the first is usually a `max_tokens` ceiling, the second is usually the dependency.

An internal operations agent. Read tools are `allowed_callers: ["code_execution_20260120"]` so that surveys of many resources happen inside the sandbox and never reach the context window; write tools are `["direct"]`. The authorisation for both is at the call site, and the team's own note in the code says what §34.7 says: the field routes, the dispatcher decides.

A support agent with a fan-out. Five to nine parallel lookups per turn, of which one or two routinely fail. The loop returns all results in a single user message with `is_error` where appropriate; a previous version returned the successes and deferred the failure, and the documented health metric `total_tool_calls / tool_call_messages` fell to 1.0 over three weeks before anyone noticed the model had stopped batching.

A migration that removed a field. A team deleted `on_behalf_of` from four tool schemas and threaded the principal through the dispatcher instead. The tool-use system prompt shrank, the four schemas dropped below the optional-parameter budget Chapter 32 measured, and an entire class of audit finding disappeared. The change was smaller than the discussion about it.

34.30 Certification Notes

Concept	Where it is examined	Domain
`is_error: true` semantics and the three error kinds	Tool result construction; recovery flow design	D2 Tool Design & MCP Integration
Instructive error strings and the retry budget	Prompt-adjacent tool design	D2, D4
Call-site validation against the unenforced subset	Schema design and safety	D2, D5
Authorising the caller, not the request	Least privilege in an agent	D1, D5
`allowed_callers` values and defaults	Tool configuration	D2
HTTP-layer versus turn-layer retry	Reliability engineering	D5 Context Management & Reliability
Idempotency keys and de-duplication windows	Agent loop architecture	D1, D5
Partial failure in a parallel batch	Orchestration	D1
`INVALID_JSON` and fine-grained streaming	Streaming and tool infrastructure	D2
Untrusted content placement	Security	D5

CERTIFICATION NOTE

Exam Tips

- **A refusal is not a tool error.** If a question describes `stop_reason: "refusal"`, no `tool_result` is the answer. Branch on `stop_reason` first, always.
- **Server tool errors are Anthropic's to handle.** You never construct a `tool_result` for a server tool, error or otherwise. But an *unresolved* `server_tool_use` block still constrains your next user message.
- **Two to three.** The documented number of times Claude retries an invalid tool call before apologising. Questions about error-string quality often hinge on it.
- `allowed_callers` **defaults to** `["direct"]` **when omitted.** Learn the two values and that both code-execution versions are interchangeable.
- **Text after** `tool_result` **blocks:** legal for a client-only turn, a 400 when a server tool is unresolved. This asymmetry is heavily examined.
- **No idempotency key on** `POST /v1/messages`**.** Documented neither way. If an answer option asserts one exists, it is wrong.
- **A client-side timeout does not cancel the generation.** This is the fact that makes the turn layer dangerous.
- `minimum`, `maximum`, `minLength`, `maxLength` **and** `multipleOf` **are not enforced by strict mode.** The SDKs strip them into the description. Expect a question that offers "add `strict: true`" as a plausible-looking fix for a range violation.

34.31 Twenty Practice Questions

1. A `tool_use` block requests a transfer of 5,000,000 minor units against a schema declaring `maximum: 1000000`, and `strict: true` is set. What happened?
2. Which of the three documented error kinds requires no action from you?
3. Your loop refuses to run one call in a parallel batch of four. What must the next user message contain?
4. A continuation `POST` times out after your tool executed. How many duplicated effects can Chapter 29's `max_retries` policy cause?
5. Where must an idempotency key be minted so that it survives a turn-layer replay?
6. What is the documented default of `allowed_callers`?
7. Why does keying idempotency on the `tool_use` id fail?
8. What single-key wrapper does the documentation prescribe for unparseable streamed tool input?
9. How many times does the documentation say Claude retries an invalid tool call before apologising?
10. Text follows the `tool_result` blocks in a user message. When is that a 400?
11. Name the four things an instructive `is_error` string should contain.
12. Which is safer to send back: a redacted stack trace or a sentence you wrote?
13. Your leak scanner passes on every `tool_result`. What have you established about error quality?
14. In what order should the gates authorise, validate and de-duplicate run, and why?
15. What does a de-duplication window of 900 seconds fail to protect against?
16. `usage` shows retries occurred and the ledger shows no duplicates. Which retry layer was exercised?
17. A model-minted idempotency key has stability 1.0. What has it become?
18. Which content block type must never be executed, even though it parses?
19. Why is `additionalProperties: false` insufficient protection against an `actor_id` field?
20. What must share a transaction with the effect, and what breaks if it does not?

34.32 Answers

1. Nothing went wrong at the API; the grammar does not enforce `maximum`. Chapter 32 measured this: the SDKs strip numeric constraints into the description and validate client-side *after* the response. *Distractors:* "strict mode failed" is wrong — strict mode did what it guarantees, which is the supported subset. "The schema is invalid" is wrong — it compiled. "Claude ignored the schema" is wrong and is the most damaging misreading, because it suggests a prompting fix for a mechanism problem.

2. Server tool errors. The documentation says Claude handles them transparently and *"you do not need to handle `is_error` results for server tools." Distractors:* tool execution errors and invalid tool use are both yours. "All three" is wrong for the same reason.

3. A `tool_result` for all four ids, the refused one carrying `is_error: true`. *Distractors:* "three results and an explanatory sentence" produces the documented 400 naming the missing id. "Three results, and mention the refusal next turn" fails for the same reason. "An empty user message" is not a continuation.

4. Zero. Re-sending a sampling request regenerates a `tool_use` block your loop has not executed; the model never executes anything. §34.19 measured 630 retries and 0 duplicates. *Distractors:* "two, one per

retry" confuses the layers; "one" assumes the retry re-runs the tool; "it depends on the tool" is true of the turn layer, not this one.

5. Upstream of the model, once per logical request, in your loop. *Distractors:* "in the tool schema" is where the *field* goes, not where the *value* comes from; "from the `tool_use` id" changes on regeneration; "let the model generate it" gives you one stability parameter governing two behaviours in opposite directions.

6. `["direct"]`**.** *Distractors:* an empty array is documented neither way and should raise; `["direct", "code_execution_20260120"]` is a valid setting but not the default, and the documentation advises choosing one rather than both; "no default, the field is required" is wrong — it is optional.

7. Because it identifies a generation, not an intent. A turn-layer replay generates a fresh id, so the key differs and the ledger sees a new operation. It deduplicates the HTTP layer, which was already safe. *Distractor:* "because ids are not unique" — they are; that is not the problem.

8. `{"INVALID_JSON": "<the unparseable input>"}`**,** serialised with a JSON library and returned as the `content` of a `tool_result` with `is_error: true`. *Distractors:* returning the raw fragment loses the signal that it is unparseable; returning `null` content tells the model nothing; retrying silently risks executing a truncated call.

9. Two to three. *Distractors:* "once" understates the budget a bad message wastes; "until `max_tokens`" confuses model behaviour with a request parameter; "indefinitely" is wrong — it apologises to the user.

10. When the assistant turn contains an unresolved `server_tool_use` **block.** For a client-only turn, text after the results is explicitly legal. *Distractor:* "always" is the common wrong answer and §34.20 reproduces both cases from the same five results.

11. The field or operation, the value that was rejected, the constraint, and what to do next including whether waiting helps. *Distractor:* "the exception type" is a diagnosis, and §34.22 measured raw exceptions at 0/40 on the fourth criterion.

12. The sentence you wrote. Both are safe once redacted; only one is useful. §34.22's `redacted_raw` arm scores 0 leaks and 0 actionable — the negative result. *Distractor:* "the redacted trace, because it has more detail" mistakes volume for instruction.

13. That they are safe. Nothing about whether they are useful. The two properties are independent and all four quadrants are occupied. *Distractor:* "that error quality is good" is exactly the inference the experiment exists to block.

14. Authorise, then validate, then de-duplicate. An unauthorised caller should never learn your limits from a validation message, and a rejected input should never consume an idempotency key. *Distractor:* "validate first, it is cheapest" optimises the wrong thing and leaks policy.

15. A replay that arrives later than 900 seconds — most commonly an operator re-running a failed job after an incident call. Size the window from the tail of the replay-delay distribution. *Distractor:* "a replay from a different process" is a store-scope problem, not a window problem.

16. The HTTP layer. Turn-layer replays would show a gap between effects and logical requests. *Distractor:* "both" cannot be concluded from those two facts alone.

17. A content digest, with a content digest's cost — 0.00 duplicates and 11.45 legitimate suppressions per thousand in $34.19. That it converges tells you it was never a separate strategy. *Distractor:* "perfect" is the trap; perfect stability is perfectly unable to tell a replay from a genuine repeat.

18. An incomplete `tool_use` block left by a `max_tokens` stop, and by extension any input you could not parse. Chapter 28 flags it; this chapter refuses to run it. *Distractor:* "a `server_tool_use` block" — you never execute those, but not because they are incomplete.

19. Because `additionalProperties: false` only stops *undeclared* properties. An `actor_id` you declared is a first-class field the model is invited to fill. The fix is to delete it, not to constrain it. *Distractor:* "add a `pattern`" constrains the shape of a claim, not its truth.

20. The idempotency-key write. Write the key first and a failed effect suppresses a retry that should have happened; write the effect first and a failed key write keeps the duplicate you were paying to avoid.

34.33 Ten Practical Exercises

1. Add `multipleOf: 100` to the payments schema, re-run `in_step`, and fix whichever side reports the gap. State which side you changed and why.
2. Write the operator-facing counterpart of `errmsg.instructive`: same failure, full upstream text, joined to the model-facing string by a request id.
3. Extend `errmsg.LEAKS` with two patterns specific to your estate — a bucket-name convention, an internal URL shape — and re-run the corpus.
4. Add a sixth arm to `errcorpus`: an instructive message that names the *tool* but not the *field*. Predict its score before running it.
5. Change `Params.p_unknown` to your own measured timeout rate and re-run the five strategies. Report duplicates per thousand with a Wilson interval.
6. Implement `dispatch(principal, tool_use_block, logical_request_id)` running gates 0 to 5, and make `partial.result_block` call it.
7. Make the ledger fail closed: if the store is unavailable, refuse the effect and return an `is_error` string saying so. Write the string.
8. Add a `caller` field to the fixtures and assert that a tool declared `["direct"]` refuses a `code_execution_20260120` caller at your call site, whatever the API does.
9. Measure the token cost of your ten longest `is_error` strings across a twenty-turn conversation, using Chapter 30's cache positions.
10. Write the test that would have caught the incident this chapter opened with, and check whether it passes against your current dispatcher.

34.34 Five Challenge Labs

1. **The shared store.** Replace the in-process `Ledger` with one backed by a store shared across processes, and demonstrate — with two processes and a forced replay — that de-duplication survives the process boundary. Report the added latency at p50 and p99.

2. **Fail closed under partition.** Extend the harness with a store that becomes unavailable for thirty seconds mid-run. Measure duplicated effects under three policies: proceed, refuse, and queue-and-retry. Publish all three, including the one that looks worst.
3. **The judge you cannot have.** §34.22's scorer is a proxy. Design — do not run — an evaluation that would measure whether Claude actually recovers better from instructive errors, using Chapter 24's pairing and power arithmetic. State the sample size, the grader and the pinning rule.
4. **Reversal cost.** Extend `idem.simulate` to price each duplicated effect with a reversal cost, and find the de-duplication window at which storage cost equals expected reversal cost. Publish the crossover and the assumption it rests on.
5. **The whole gate on a real catalogue.** Take Chapter 33's five-tool support catalogue, add effects to two of them, and run all five gates over forty requests. Report the four dashboard metrics from §34.21 and say which gate fired most.

34.35 Ten Interview Questions

1. A colleague sets `max_retries=0` on a payments workload "so we never double-charge". What do you say?
2. Where does an idempotency key come from, and what is wrong with each of the three sources you did not choose?
3. `strict: true` is on and a value exceeded the declared maximum. Walk me through why, without saying "a bug".
4. What is in a good `is_error` string, and what is in a bad one that looks good?
5. Why is a redacted stack trace not an improvement?
6. Your tool needs to know which user it is acting for. Where does that come from?
7. Explain the difference between `allowed_callers` and authorisation.
8. One call in a parallel batch of six fails. Describe the next user message precisely.
9. How would you measure duplicated side effects in a system you have just inherited?
10. What is the relationship between a de-duplication window and a prompt cache TTL?

34.36 Summary

This chapter took the seam between the model's request and your effect and put five gates in it, because three earlier chapters had between them deferred every question about failure to this one. Chapter 31 built the loop that runs tools; Chapter 32 built the schema that shapes their inputs and measured, precisely, what that schema does not enforce; Chapter 33 established that everything the model can see about a tool is a prompt. Chapter 34's contribution is that the same is true of everything the model can see about a tool's *failure*, and that the gap between "the schema validated" and "the effect is safe" is wider than it looks and is measurable.

The measurement of that gap is the first of the chapter's four results. A payments tool declaring sixteen constraints has thirteen enforced by the grammar, and the three misses are the floor, the ceiling and the length — every one of them a safety check. Twenty-four realistic inputs, all conformant, produce nine

call-site rejections and one transfer ten times the declared cap. The remedy is not a longer schema but a validator **derived from the audit**, so that the set of things re-checked at the call site is by construction the set the API does not check. `in_step` returns both sets, not a boolean, so that when it eventually disagrees you can see which side moved.

The second result is the one that started the chapter. Two things are called retry and they behave differently: 0.00 duplicated effects per thousand at the HTTP layer, structurally, because the model never executes anything and a re-sent request regenerates a `tool_use` block nobody ran; and 31.65 per thousand at the turn layer, because the effect happens before the post that failed. The remedy is a key, and the interesting part is which key. The `tool_use` id is worthless — it deduplicates the layer that was already safe. A content digest works and suppresses 11.45 legitimate operations per thousand. A model-minted key is one coin with two faces, and the sweep from stability 0 to 1 shows duplicates and suppressions trading off exactly, arriving at the content digest's numbers at the top. Only a key minted upstream of the model, once per logical request, is clean on both axes — and the reason is structural rather than empirical: it is the only source that knows the difference between the same intent arriving twice and two intents that look alike.

The third result is about words. Five ways of writing the same forty failures, scored on four criteria and on disclosure. The raw exception, which feels like the informative choice, names the field sixteen times out of forty and says what to do next zero times out of forty — a diagnosis with no instruction in it — while leaking internal hostnames, connection strings or private addresses in thirty-six. Redacting it removes the danger and adds no help: nought leaks, nought actionable, published as a negative result because a team that ships only the leak scanner will believe it has fixed something. And the most dangerous arm is the most conscientious one, an instructive message that also quotes the upstream text for the operator's benefit: forty out of forty actionable, thirty-six out of forty leaking. All four quadrants occupied, two independent properties, two checks required. The economics are not close — thirty-four extra tokens at $0.000170 against $0.026500 for the round trip they prevent, a ratio of 156, before counting the two-to-three retries the documentation says an unfixable error will spend.

The fourth is smaller and is mostly about discipline. A call you refused is still a `tool_result`; a call you could not parse is still a `tool_result`, wrapped by a JSON library rather than by concatenation; and text after the results is legal for a client-only turn and a 400 when a server tool is unresolved, which the harness reproduces from the same five results by changing one block. Underneath all of it sits Chapter 25's rule, made concrete: authorise the caller, from your session, and never the request, because the request was written by a component that has read untrusted text — and `allowed_callers`, whose own documentation reaches for *guides* on one page and lists a `tool_not_allowed` error on another, is routing rather than a boundary until that is resolved.

KEY TAKEAWAYS

1. `is_error: true` **addresses the model, not your logs.** It is a prompt written at the worst moment and read immediately.
2. **A raw exception is a diagnosis, never an instruction** — 24/40 on stating the constraint, 0/40 on saying what to do next.
3. **Safety and usefulness are independent properties of an error string.** Redaction scores 0 leaks and 0 actionable; check both or you know neither.
4. **A grammar enforces shape; safety lives at the call site** — and all three of Chapter 32's misses were range checks.
5. **Derive the validator from the audit.** Two hand-maintained lists of constraints are two lists that will disagree.
6. **Authorise the caller, from your session, never the request.** Nothing identity-shaped belongs in an `input_schema`.
7. `allowed_callers` **routes; your dispatcher decides.** Two documentation pages use different verbs, so take the conservative reading.
8. **Retrying the call cannot duplicate an effect; retrying the turn can** — 0.00 against 31.65 per thousand, same failure rate, same word.
9. **An idempotency key must be minted upstream of the model, once per logical request.** Any key the model can influence trades duplicates against suppressions along one line.
10. **Every** `tool_use` **id gets a** `tool_result` — including the call you refused, the call you could not parse, and the call you never ran.

34.37 Glossary

Term	Meaning
`allowed_callers`	Optional tool property restricting which context may invoke a tool; values `direct` and `code_execution_20260120`, defaulting to `["direct"]`
Call site	The point in your code where the effect happens; Chapter 25's action layer
Call-site validation	Re-checking the model's input against your original schema, immediately before the effect
`caller`	Field on a `tool_use` block identifying how the call was invoked
Content digest	An idempotency key derived from the tool name and arguments; cannot distinguish a replay from a genuine repeat
De-duplication window	How long your store remembers a key; a bet on the tail of the replay-delay distribution
Error contract	The agreement about how failure is communicated to the model; unlike the input schema, nothing validates it
Fail closed	Refusing the effect when the control protecting it is unavailable
HTTP-layer retry	Re-sending one `POST /v1/messages`; re-samples the model, never re-executes a tool

Term	Meaning
Idempotency	Performing an operation more than once has the same observable effect as performing it once
`INVALID_JSON`	The documented single-key wrapper for unparseable streamed tool input, returned with `is_error: true`
`is_error`	Optional boolean on a `tool_result` marking the call as failed
Instructive error	A message naming the field, the value, the constraint and the next step
Key stability	The probability a model re-mints the same key for the same arguments; governs duplicates and suppressions in opposite directions
Logical request	One intent that entered your system; the correct scope for an idempotency key
Turn-layer retry	Re-running a whole assistant turn; re-executes your tool by construction
Unrun call	A `tool_use` you chose not to execute; still requires a `tool_result` with `is_error: true`

34.38 Further Reading

Primary sources, verified 3 August 2026 at `platform.claude.com/docs/en`:

- `agents-and-tools/tool-use/handle-tool-calls` — the `tool_result` shape, the formatting requirements, the untrusted-content warning, and the three error kinds with the instructive-error guidance. The error section is inside collapsed accordions in the rendered page; the Markdown source at the same path with a `.md` suffix returns it in full.
- `agents-and-tools/tool-use/troubleshooting-tool-use` — symptom-to-fix tables, including the exact 400 messages for missing `tool_result` blocks and unresolved server tools, and the note that JSON escaping differs across model versions so you must parse rather than string-match tool inputs.
- `agents-and-tools/tool-use/tool-reference` — the optional tool properties table, the `allowed_callers` values, and the interchangeability of the two code-execution versions.
- `agents-and-tools/tool-use/programmatic-tool-calling` — the `caller` response shape, the `tool_not_allowed` error, and the model compatibility table that does not yet list Opus 5.
- `agents-and-tools/tool-use/fine-grained-tool-streaming` — `eager_input_streaming`, the accumulation contract, and the `INVALID_JSON` wrapper with the instruction to build it with a JSON library.
- `agents-and-tools/tool-use/strict-tool-use` — what `strict: true` guarantees, and by omission what it does not.

Where to go next in this book. Chapter 35 closes Part 6 by taking these gates up to orchestration across many tools and turns, where a failure is a branch rather than an ending. **Part 7 (Chapters 36–40)** introduces MCP, where the tools, the schemas and the error strings were written by someone else and this chapter's gates become the only thing you control. **Chapters 47–51** build the agent loop properly, including reflection and human-in-the-loop approval for the calls this chapter merely refuses. **Chapter 56** catalogues failure modes at system scale and **Chapter 57** builds the tracing that turns "31.65 duplicates per thousand" into an alert. **Chapter 58** takes the untrusted `tool_result` channel as its subject.